CASE-STUDY PRACTICE | CAPSTONE 7 | STUDY EDITION

ETA Prediction: Correcting a Routing Engine in Milliseconds

Scenario questions on predicting the residual of a routing engine's arrival time, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on data, a model or a serving fleet: figures are the sources' or arithmetic from stated inputs, and request rates, host counts, hardware speeds and table sizes are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: post-processing, different consumers and the sizing 6

2 Data and features: discretize, geohash and hash 10

3 The model: linear attention, calibration, the loss and the results tables 14

4 Serving, evaluation and operations 19

Sources and confidence 24

Every question, by part

Part 1 Framing the problem: post-processing, different consumers and the sizing 6

01 Correct the routing engine, or replace it? Framing 6

02 Which error matters to whom? Objective 7

03 How much traffic is there? Estimation 8

04 Where do the 3.25 milliseconds go? Latency 9

Part 2 Data and features: discretize, geohash and hash 10

05 Why bin the continuous features? Features 10

06 How big would the location table be? Geospatial 11

07 Hashing helped here. Why not at Google? Design choice 12

08 Review the location hashing Code review 13

Part 3 The model: linear attention, calibration, the loss and the results tables 14

09 Choose the attention variant Architecture 14

10 One model for rides and deliveries Architecture 15

11 Build the loss Loss 16

12 Read the results tables Evaluation 17

13 Review the loss function Code review 18

Part 4 Serving, evaluation and operations 19

14 Trace one ETA request Serving 19

15 Two sources, two figures Sources 20

16 Compare with the old model, and region by region Evaluation 21

17 Watch for drift Monitoring 22

18 Risks, and ten times the traffic Risk 23

The case in one page

A summary of the case study to read before the questions.

Problem	A fast, accurate arrival time for rides and deliveries, when the route is not known in advance and different consumers need different kinds of error.
Core idea	A routing engine gives a first ETA from the best path. A neural network predicts the residual against the observed arrival time and adds it back.
Model	Discretize and embed every feature, one linear-attention layer, and a decoder with a bias per request type. 141.2 million parameters, 0.5 million dense (source-confirmed).
Loss	Asymmetric Huber, with delta for outlier robustness and omega for the cost of late against early. The values are not published (source-confirmed).
Latency	Median 3.25 ms, 95th percentile 4 ms in production; the blog requires a few milliseconds at most (source-confirmed).
Results	MAE improves 7.83% over the routing engine ETA offline; online against the production XGBoost 2.91% for delivery and 2.66% for rides (source-confirmed).
First to break	Serving time and embedding memory, not arithmetic: the dense part is about 0.5 million weights.
Easy-to-miss traps	Two parameter figures disagree. By operation count the attention layer is small. Hashing helped here after the CTR paper rejected it. Table 2 and Table 7 have different baselines.

Numbers to know

Each figure carries the tag that says how it was obtained. The sources give request volume only in words, so rates, hosts, hardware speeds and table sizes are illustrative.

Quantity	Value	Evidence
Training data	About 1.4 billion ETA requests over 14 days for training and validation, plus 7 days for test; requests over 2 hours removed	source-confirmed (the paper)
Rate anchors	1.4 billion ÷ (14 × 86,400) = about 1,160 a second if unsampled; 2 billion a week = 3,300 a second, peak 13,200 at an assumed 4 times	illustrative; recomputed
Latency	Median 3.25 ms; p95 4 ms; blog requirement: a few milliseconds at most	source-confirmed (the paper, the blog)
Little's law	13,200 × 0.00325 = 43 in flight; about 11 hosts of 4 cores at full use; about 36 at 30%	illustrative; recomputed
Parameters	ResNet 141.6 million (1.2 dense); HammockNet 140.8 (0.5); DeeprETANet 141.2 (0.5)	source-confirmed (Table 4)
Disagreeing figures	Blog: 0.25% touched = 353,000; Table 4 dense part 0.5 million = 0.35%; the two cannot both hold	source-confirmed; recomputed
Dense compute	0.5 million weights = 1 million FLOPs = 0.05 ms at 20 billion FLOPs a second per core	illustrative; recomputed
Attention cost	40 features of 8 dimensions: 12,800 against 2,560 (5 times); 200 features: 320,000 against 12,800 (25 times)	source-confirmed (blog example); recomputed
Geohash cells	2^(5u): u = 4: 1,048,576; 5: 33.6 million; 6: 1.07 billion; 7: 34.4 billion	recomputed
Location tables	Exact at u = 7: about 550 GB; two hashes into 1 million bins: 32 MB; 12 hashed features: 384 MB	illustrative; recomputed
Binning information	Ten speeds: equal-width 0.92 bits; quantile 1.97 bits	illustrative; recomputed
Table 2	XGBoost 5.07/1.79/5.98; ResNet 7.39/6.19/7.27; HammockNet 7.58/6.12/7.56; DeeprETANet 7.83/6.25/7.99 (MAE/p50/p95 %)	source-confirmed (the paper)
Table 3	Delivery 7.46/2.26/11.05; rides 7.86/6.25/7.62	source-confirmed (the paper)
Table 7	Over production XGBoost: delivery 2.91/4.74/3.92; rides 2.66/4.15/3.23	source-confirmed (the paper)
Table 6 and gaps	DeeprETANet MAE: NYC 7.20, LA 6.80, London 4.80, Mexico City 6.60; lead over XGBoost 3.1, 2.0, 3.4, 2.7 points	source-confirmed; recomputed
Loss example	Delta 60, omega 0.3, error 120 s: base 5,400; late 3,780; early 1,620 (2.3 times)	illustrative; recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Base estimate	Routing engine ETA plus a learned residual	An end-to-end model if the base is unreliable and the coupling is costly	source-confirmed; reasoned
Features	Discretize and embed everything	Raw continuous values for a tiny model	source-confirmed
Geospatial encoding	Multi-resolution geohash with two independent hashes	Exact indexing if memory is plentiful and cells are dense	source-confirmed
Interaction layer	Linear self-attention	Standard attention if its measured cost fits the budget	source-confirmed; reasoned
Heterogeneity	A bias per request type	A multi-head or expert model if latency allows	source-confirmed
Loss	Asymmetric Huber	A quantile loss if one quantile is the only target	source-confirmed; reasoned
Serving	CPU hosts with 4 cores	GPU if the dense part grows a lot	source-confirmed; reasoned
Launch gate	A two-week global shadow test against the production model	A shorter test for a small change	source-confirmed

PART 1

Framing the problem: post-processing, different consumers and the sizing

Why correct a routing engine instead of replacing it, what different consumers need from an estimate, and where a 3 millisecond model spends its time.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Routing engine ETA (RE-ETA)	The arrival time a route planner computes by finding the best path with a shortest-path algorithm and adding up the expected time of each road segment.
Actual time of arrival (ATA)	The observed time. For a pickup it runs from the driver accepting the request to the start of the trip. For a drop-off it runs from the start of the trip to its end.
Residual and post-processing	The residual is ATA minus RE-ETA. A model that predicts it and adds it back to the routing engine's estimate is a post-processing model.
MAE, p50 and p95	Mean absolute error, and the 50th and 95th percentiles of the absolute error. The mean says how large errors are on average, and p95 says how bad the worst few are.
Mean, quantile and point estimate	A point estimate is one number given as the prediction. Different uses want different ones: the mean for fares, a higher quantile when a late arrival is costly.
Little's law	The average number of requests in the system equals the arrival rate times the time each spends there.
Shadow test	Running a new model on live traffic and logging its predictions without showing them to users, so it can be measured before launch.
Multiply-add (MAC)	One multiplication plus one addition, the unit used to count a neural network's work. A layer with a inputs and b outputs costs a times b per example.
Where to read	Hu, Binaykiya, Frank, Cirit: DeeprETA: An ETA Post-processing System at Scale (arXiv:2206.02127) | Uber Engineering: DeepETA: How Uber Predicts Arrival Times Using Deep Learning (blog, February 2022)

01 Correct the routing engine, or replace it?

FRAMING

SCENARIO

A new engineer proposes dropping the routing engine ETA and training one model to predict the arrival time from trip features alone. The paper's design predicts a residual on top of the routing engine ETA. Argue both sides, and say what the sources prove.

HOW TO THINK ABOUT IT

List what the routing engine gets right, what it cannot know, and whether the sources test the alternative.

Q What does the routing engine do well?

A It uses map data and real-time traffic, finds the best path, and sums segment times. The paper says modern routing engines even account for accidents and weather per segment (source-confirmed).

- A model trained from scratch would have to relearn that structure from trip data.

Q What can it not know?

A The route a driver will actually take, human mistakes on difficult stretches, and differences between tasks such as driving to a restaurant and driving to a rider. Its objective is the best path, not an end-to-end ETA, and the ends of a trip add noise such as finding parking (source-confirmed).

- These are the gaps the residual model is meant to learn.

Q What practical reasons does the blog give for post-processing?

A It is generally easier to take in new data sources and changing business requirements by updating the post-processing model than by refactoring the routing engine.

- Uber's earlier production model was also a post-processing model: a group of XGBoost models, one per mega-region.

Q What do the sources prove?

A The hybrid beats the unaided routing engine and several baseline regression models. They do not compare against a model that ignores the routing engine, so the choice of correcting over replacing is argued, not tested.

- The paper also says it evaluated only Uber's own routing engine.

Q Reframe it.

A When a good base system exists, learning its error is cheaper than relearning its job. Say which comparison your evidence covers, and which it does not.

Basis: The routing engine description, its four shortcomings and the blog's practical reason are from the paper and the blog (source-confirmed); the statement about what is not compared is a reading of the sources.

02 Which error matters to whom?

OBJECTIVE

SCENARIO

The fares team wants the best mean error. The delivery team wants the 95th percentile kept down. A manager asks for one model that does both. Explain what the paper does, using its Table 3.

HOW TO THINK ABOUT IT

Read what each consumer needs, then read the gains by request type.

Q What do the consumers need?

A For fares, the mean ETA error matters. For delivery, both the mean absolute error and the 95th percentile matter, since extreme errors give a bad experience. User-facing ETAs may call for a set of quantiles or expectiles (source-confirmed).

- The paper says that optimizing for a single point estimate is not ideal.

Q What does the paper do about it?

A It uses an asymmetric Huber loss with two parameters, one for robustness to outliers and one for the cost of underprediction against overprediction, so the point estimate can be tailored to business goals (Question 11).

- It reports MAE, p50 and p95 for every model.

Q What does Table 3 show?

A DeeprETANet's improvement over the routing engine ETA is MAE 7.46%, p50 2.26% and p95 11.05% for delivery, and 7.86%, 6.25% and 7.62% for rides (source-confirmed).

- Rides gain most at the median, and delivery gains most at the tail. One model does not move all three the same way.

Q What would you tell the manager?

A One shared model can serve both, and its tail and median gains differ by request type. Pick the metric each team signs off on, and check each one by type before launch.

- The paper does not say how the loss parameters were chosen.

Q Reframe it.

A Different consumers need different errors from the same number. Name the metric each one cares about, and report them separately.

Basis: The consumer needs, the loss and Table 3 are from the paper (source-confirmed).

03 How much traffic is there?

ESTIMATION

SCENARIO

The paper says Uber has billion-level requests a week and that this is the highest-QPS model at Uber. It also describes its training data. Size the traffic and the fleet, being clear which anchor is which.

HOW TO THINK ABOUT IT

Convert the training data to a rate, then use Little's law with the paper's latency.

Q What does the training data give?

A About 1.4 billion ETA requests over a 14-day training and validation window. 1,400,000,000 ÷ (14 × 86,400) = about 1,160 requests a second (source-confirmed counts).

- The paper does not say whether the data is a sample, so this is a lower bound on the production rate.

Q Is that consistent with billion-level a week?

A 1.4 billion over two weeks is under a billion a week. Either the data is a sample, or billion-level is loose.

- An alternative anchor is 2 billion a week (illustrative): 3,300 a second on average, and 13,200 at an assumed peak ratio of 4.

Q How many requests are in flight?

A By Little's law: rate × time. At 13,200 a second and the paper's 3.25 ms median, about 43 requests at once.

- Production hosts have 4 cores each (source-confirmed).

Q How many hosts?

A 43 ÷ 4 cores = about 11 hosts at full use, and about 36 at 30% use.

- One user request may trigger several model calls, which would multiply this.

Q Reframe it.

A Anchor on a number the source gives, say what it could be a sample of, and size from there. The conclusion should survive a change of anchor.

Basis: The 1.4 billion, the window, the latency and the 4 cores are from the paper (source-confirmed); the 2 billion a week, the peak ratio and the utilization are illustrative; arithmetic recomputed.

04 Where do the 3.25 milliseconds go?

LATENCY

SCENARIO

The model has 141.2 million parameters and answers in a median 3.25 ms. A teammate says: the arithmetic of a 141-million-parameter model must be the cost. Check it.

HOW TO THINK ABOUT IT

Count the dense multiply-adds, then ask which parameters a request actually reads.

Q How many parameters are dense?

A 0.5 million of 141.2 million are outside the embeddings (Table 4). The rest are lookup tables (source-confirmed).

- The paper says the embeddings contribute minimally to latency because an embedding lookup takes constant time.

Q What does the dense work cost?

A 0.5 million weights is about 1 million FLOPs per request. At an assumed 20 billion FLOPs a second per core that is 0.05 ms.

- It is about 1.5% of the 3.25 ms median.

Q Then what takes the time?

A Not the arithmetic. Likely candidates are assembling the features, the call to the prediction service, queueing and any batching. The sources do not break the time down.

- That is this document's reading. Measure it on your own system.

Q What does the attention layer add?

A The blog's example: 40 features of dimension 8 gives 12,800 multiply-adds for standard attention against 2,560 for linear attention. At the same assumed speed, 12,800 takes under a microsecond.

- The paper says standard attention was too slow, and gives no timing. Treat that as the authors' finding and measure your own shapes.

Q Reframe it.

A Count the operations before blaming them. A model can have many parameters and very little arithmetic, and the latency then lives elsewhere.

Basis: The parameter counts, the latency and the blog's example are from the sources (source-confirmed); the 20 billion FLOPs a second is illustrative; the breakdown of the rest of the latency is this document's reading.

PART 2

Data and features: discretize, geohash and hash

Why a network does better on binned inputs, what a geohash resolution means for table size, and when hashing helps.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART	
Quantile bucketing	Splitting a feature's values into bins that each hold about the same number of examples. The bin edges follow the data's distribution.
Equal-width bucketing	Splitting the range of a feature into bins of the same width. With a long tail most examples can land in a few bins.
Entropy (bits)	A measure of how much information a bin label carries. For a fixed number of bins it is highest when the bins are equally likely.
Geohash	A string that names a grid cell on the globe. A geohash of length u carries 5u bits, so the number of cells grows exponentially with u.
Exact indexing	Giving every distinct value, here every grid cell, its own embedding row. It is accurate and takes the most space.
Multiple feature hashing	Mapping each cell to bins with several independent hash functions and learning an embedding for each bin, so a collision in one hash is offset by the others.
Embedding table	A lookup from an index to a learned vector. Most of this model's parameters are in such tables.
Stable hash	A hash function that gives the same output in every process and every run. Training and serving must use the same one.
Where to read	Hu, Binaykiya, Frank, Cirit: DeeprETA: An ETA Post-processing System at Scale (arXiv:2206.02127) | Uber Engineering: DeepETA: How Uber Predicts Arrival Times Using Deep Learning (blog, February 2022)

05 Why bin the continuous features?

FEATURES

SCENARIO

The paper bucketizes continuous features such as speed into 256 quantiles before embedding them, and says this beat using the raw value. It also says quantile buckets beat equal-width ones. A skeptic says a network can learn any function of a number, so binning only throws information away. Respond with a worked example.

HOW TO THINK ABOUT IT

Bin a small skewed sample both ways and compare the information in the bin labels.

Q What does the paper report?

A Discretizing and embedding all the features gave a significant accuracy lift over continuous features. Speed in 256 quantiles was better than using speed directly, and quantile buckets beat equal-width buckets (source-confirmed).

- The blog's explanation: a network with bucketized features does not spend parameters learning to partition the input space.

Q Bin ten speeds two ways.

A Speeds: 5, 8, 9, 10, 11, 12, 13, 15, 40, 120. Equal-width bins of 30 over 0 to 120 hold 8, 1, 0 and 1 values. Quantile bins hold 3, 2, 2 and 3.

- Most of the data sits in one equal-width bin, where the labels cannot tell 5 from 15.

Q Compute the information.

A Equal-width: about 0.92 bits. Quantile: about 1.97 bits, close to the 2-bit maximum for 4 bins.

- This is the paper's argument: for a fixed number of buckets, quantile buckets convey the most information about the original value.

Q What does binning cost?

A Precision inside a bin: two speeds in one bin look the same. With 256 bins it is small, and the lookup replaces learning where to cut.

- A skewed feature such as speed or distance benefits most.

Q Reframe it.

A A model can in principle learn what you hand it, and it pays in capacity and data. Give it a representation that already carries the information.

Basis: The findings and explanations are from the paper and the blog (source-confirmed); the ten speeds and the bit counts are illustrative arithmetic (recomputed).

06 How big would the location table be?

GEOSPATIAL

SCENARIO

The model encodes origin and destination as geohashes at lengths 4, 5, 6 and 7. A teammate wants an exact embedding for every cell. Size it, and size the hashed alternative.

HOW TO THINK ABOUT IT

Count cells at each length, then bytes for an exact table and for a hashed one.

Q How many cells?

A A length-u geohash has 5u bits, so 2^(5u) cells: u = 4 is 1,048,576; u = 5 is 33.6 million; u = 6 is 1.07 billion; u = 7 is 34.4 billion.

- The paper says the number of cells grows exponentially with resolution and the average data per cell falls in proportion.

Q What is an exact table at u = 7?

A 34.4 billion rows. With an assumed 4 dimensions in float32, 16 bytes a row, that is about 550 GB for one feature at one resolution.

- The paper says exact indexing takes the most space.

Q What does hashing cost?

A With two independent hash functions into 1 million bins each (assumed), 2 × 1,000,000 × 16 bytes = 32 MB.

- The paper hashes the origin, the destination and the pair, at every resolution: 3 features × 4 resolutions = 12 hashed features, about 384 MB at this assumed size, in a model of 141 million parameters.

Q What is lost?

A Different cells can share a bin. The paper says that independent hashes let the network combine information from several buckets and undo the harm of a collision in one.

- The finest grid is also sparse, so each exact cell has few examples to learn from. Sharing a bin can help.

Q Reframe it.

A Size a table before you build it. When the key space grows exponentially, the question is how much sharing the data can tolerate.

Basis: The geohash structure, the exponential growth and the hashing scheme are from the paper (source-confirmed); the 4 dimensions, float32 and 1 million bins are illustrative; arithmetic recomputed.

07 Hashing helped here. Why not at Google?

DESIGN CHOICE

SCENARIO

The CTR paper in this series reports that Google tried feature hashing and rejected it. Uber's paper uses hashing and says removing it hurts. A reviewer says the two results contradict each other. Resolve it.

HOW TO THINK ABOUT IT

Read the ablation, then compare the two situations: how sparse the data is and what a collision costs.

Q What does Uber's ablation say?

A Removing feature hashing, which means indexing the geohash string exactly, lowered MAE improvement from 7.83% to 7.41% and p50 improvement from 6.25% to 5.10%. The p95 improvement stayed at 7.99% (source-confirmed).

- The paper reads this as richer geospatial embeddings helping typical cases and not extreme errors.

Q What does the blog add?

A Single-hash feature hashing saved space with the same or slightly worse accuracy than exact indexing, because collisions lose information. Multiple feature hashing gave the best accuracy and latency while still saving space.

- So the gain came from using several independent hashes, not from hashing alone.

Q Why might it differ from Google's result?

A The inputs differ. Google's hashed features were very high-cardinality categorical features, and it could not shrink below several billion buckets without loss. Uber's cells at fine resolutions are sparse, and neighboring cells carry similar information, so sharing a bin can help.

- That is an explanation consistent with both papers, and neither paper tests it.

Q What would you do?

A Run the exact-against-hashed comparison on your own data and measure it by tail and median separately.

- Do not carry a rejection or an adoption from another paper to a different feature.

Q Reframe it.

A Two opposite results can both be right when the situations differ. Find the difference before choosing a side.

Basis: The ablation figures and the blog's statements are from the sources (source-confirmed); the comparison with Google's result refers to the CTR case study; the explanation is reasoned and untested.

08 Review the location hashing

CODE REVIEW

SCENARIO

A colleague writes this location feature. Offline accuracy is fine, then after deployment the live predictions are worse than offline, and two serving hosts disagree about the same request. Find the faults.

```
def location_bins(lat, lng, resolutions=(4, 5, 6, 7), bins=1_000_000):
    out = []
    for u in resolutions:
        cell = f"{round(lat, u)},{round(lng, u)}"
        out.append(hash(cell) % bins)
    return out
```

HOW TO THINK ABOUT IT

Check what round does, whether the hash is stable across processes, and what the paper hashes.

Q Does it make geohash cells?

A No. round(lat, u) rounds to u decimal places, which is a different grid. u = 4 is about 11 meters and u = 7 about a centimeter, so it does not give coarse to fine cells.

- A geohash of length u has 5u bits, and the resolutions 4 to 7 are coarse to fine in that sense.

Q Is the hash stable?

A Python's built-in hash of a string is randomized per process. Two processes give different bins for the same location, so training and each serving host disagree.

- The paper uses MurmurHash3 with independent seeds. Use a stable hash everywhere.

Q What else is missing?

A A single hash, so a collision cannot be undone. The paper hashes each cell with two independent functions, and it hashes the origin, the destination and the origin-destination pair.

- Using the origin and the destination separately and together captures the pair.

Q State the fix.

A Compute a real geohash of length u, hash it with two seeds of a stable hash, and do it for origin, destination and the pair at each resolution.

- Test: the same coordinates give the same bins in two separate processes, and nearby points share coarse cells.

Q Reframe it.

A A feature must be computed identically in training and serving. Check determinism across processes before anything else when serving disagrees with offline results.

REFERENCE CODE

```
import mmh3   # MurmurHash3, stable across processes

def location_bins(origin, dest, geohash, resolutions=(4, 5, 6, 7), bins=1_000_000):
    # geohash(lat, lng, u) returns the length-u geohash string for a point
    out = []
    for u in resolutions:
        o = geohash(*origin, u)
        d = geohash(*dest, u)
        for key in (o, d, o + "|" + d):                  # origin, destination, pair
            out.append((mmh3.hash(key, 1) % bins, mmh3.hash(key, 2) % bins))
    return out
```

Basis: The scheme the fix follows is from the paper (source-confirmed); the faults are standard and the fix was checked by hand and not run.

PART 3

The model: linear attention, calibration, the loss and the results tables

Choosing an attention variant by what it costs, adjusting for request types, building a loss that fits the business, and reading the tables honestly.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Self-attention over features	Each feature is a vector, and each is re-weighted by how much every other feature matters for it. Here the vectors are features such as origin and time of day, not words.
Positional encoding	Extra information that tells a Transformer where each token sits. It is dropped here because feature order carries no meaning.
Linear attention	A rewrite of attention whose cost grows with L times d squared, instead of L squared times d, for L features of dimension d, by avoiding the L by L matrix.
Calibration (bias) layer	A learned bias for each request type, added to the output, so one shared model can correct for mean differences between types.
Huber loss	Squared error for small errors and a linear penalty for large ones, so outliers pull less. A parameter delta sets where it switches.
Asymmetric loss	A loss that weights underprediction and overprediction differently, with a parameter omega, so the estimate leans toward the cheaper mistake.
Relative improvement	The drop in an error measure against a baseline, divided by the baseline's error. Different baselines make tables incomparable.
Ablation	Removing one component and re-measuring, to see what it contributes.
Where to read	Hu, Binaykiya, Frank, Cirit: DeeprETA: An ETA Post-processing System at Scale (arXiv:2206.02127) | Uber Engineering: DeepETA: How Uber Predicts Arrival Times Using Deep Learning (blog, February 2022)

09 Choose the attention variant

ARCHITECTURE

SCENARIO

The paper tried standard Transformer attention, found it too slow, and chose linear attention. A reviewer asks for the cost formulas, when the saving matters, and whether the claim is checkable. Answer.

HOW TO THINK ABOUT IT

Compute both costs at the blog's size and at a larger size, and compare with the dense layer.

Q What are the costs?

A For L features of dimension d, standard attention is O(L^2 × d) because it builds an L by L matrix. The linear version is O(L × d^2). The paper says it is much faster when L is much larger than d, which is common here (source-confirmed).

- The saving appears whenever L is above d.

Q Compute the blog's example.

A L = 40 features, d = 8: standard 40 × 40 × 8 = 12,800, linear 40 × 8 × 8 = 2,560, which is 5 times fewer.

- At an assumed 20 billion multiply-adds a second per core, 12,800 takes under a microsecond.

Q When does it matter?

A At L = 200 and d = 8: standard 320,000, linear 12,800, which is 25 times fewer. The saving grows with the number of features.

- The dense layer of about 0.5 million weights is already larger than the attention cost at the blog's size.

Q Is the paper's claim checkable?

A Not from its numbers. It says the original layer gave good accuracy and was too slow for online serving, and gives no timing. By operation count, attention is small at the blog's size.

- The authors may have used larger shapes or seen implementation overheads. Treat it as their finding, and measure your own.

Q What do you lose?

A Linear attention is an approximation of the full pairwise matrix, so it can represent fewer interaction patterns. The paper does not report the accuracy difference.

- Hence the decision rule: measure accuracy and latency on your shapes.

Q Reframe it.

A Do the cost arithmetic at your own sizes. A published complexity gain can be real and still irrelevant at the scale you run.

Basis: The complexity formulas, the blog's example and the paper's statement are from the sources (source-confirmed); the 20 billion multiply-adds a second and the L = 200 case are illustrative; arithmetic recomputed.

10 One model for rides and deliveries

ARCHITECTURE

SCENARIO

Rides and deliveries have quite different distributions. The paper adds a bias per request type. A colleague proposes just adding the request type as another feature. Compare, with the paper's results.

HOW TO THINK ABOUT IT

Say what the bias layer does, what the alternatives cost, and what the ablation shows.

Q What is the layer?

A The decoder is a fully connected layer plus a bias for each request type, learned from a one-hot encoding of the type. It adjusts the raw prediction for mean-shift differences across types (source-confirmed).

- Segments include delivery against rides, long against short trips, pickup against drop-off and mega-regions, according to the blog.

Q Show the idea with numbers.

A Suppose the true mean residual is +30 seconds for rides and +90 seconds for deliveries (invented). A shared output alone averages them and is wrong by about 30 seconds for each type. A bias per type absorbs the 60-second gap.

- The rest of the network learns patterns common to both.

Q What are the alternatives?

A Embedding the type so the interaction layer sees it, which the paper also implemented. A multi-head or mixture-of-experts structure, which the paper tried and found did not meet the latency requirement (source-confirmed).

- The blog says the bias layer performed better than simply adding segment features, and was chosen over a multi-task decoder because of latency.

Q What does the ablation show?

A Removing calibration lowered the MAE improvement from 7.83% to 7.73% and p50 from 6.25% to 6.12%.

- A small gain at almost no cost. It handles mean shifts and not differences in shape.

Q Reframe it.

A When subpopulations differ mainly in level, give each a bias before giving each a model. It is the cheapest correction that works.

Basis: The layer, the alternatives and the ablation are from the paper and the blog (source-confirmed); the 30 and 90 second means are invented for illustration.

11 Build the loss

LOSS

SCENARIO

A product owner says: being late costs more than being early, and delivery cares about the tail. Explain the paper's loss, compute it for an example, and say what the paper does not publish.

HOW TO THINK ABOUT IT

Write the two cases of the loss, then price an error of 30 seconds and of 120 seconds in each direction.

Q What is the loss?

A A Huber loss with parameter delta: squared error (half the error squared) below delta, and delta × error minus half delta squared above it. Then a weight: omega when the actual time is less than the prediction, and one minus omega otherwise (source-confirmed).

- Delta sets robustness to outliers. Omega sets the relative cost of underprediction against overprediction.

Q Which direction is which?

A Actual less than predicted is overprediction: the rider arrived earlier than told. Actual greater than predicted is underprediction: the rider arrived later than told. Late is worse, so underprediction gets the larger weight, which means omega below 0.5.

- The paper says being a minute late is worse than being a minute early.

Q Compute with delta = 60 and omega = 0.3 (illustrative).

A Error 30 s (inside delta): half × 900 = 450, so 315 if late and 135 if early. Error 120 s: 60 × 120 − 1,800 = 5,400, so 3,780 if late and 1,620 if early.

- Late costs 0.7 ÷ 0.3 = 2.3 times as much as early.

Q What does that do to the prediction?

A It pushes estimates later, since a high estimate is cheaper than a low one when omega is below 0.5. This is a reading of the loss, not a result in the paper.

- The model optimizes whatever asymmetry it is given.

Q What is not published?

A The values of delta and omega. A business judgment sets omega, and the paper does not say how much worse late is than early.

- Treat the numbers above as an illustration.

Q Reframe it.

A Put the business cost of each kind of mistake into the loss, and say who chose the numbers.

Basis: The loss form, its two parameters and the late-against-early statement are from the paper (source-confirmed); delta = 60 and omega = 0.3 are invented; the effect on predictions is reasoned; arithmetic recomputed.

12 Read the results tables

EVALUATION

SCENARIO

A slide says DeeprETA improves ETA error by 7.83% and also by 2.9%. A listener asks which is right. Use the tables to explain, and say what else the tables show.

HOW TO THINK ABOUT IT

Name the baseline in each table first.

Q What does 7.83% measure?

A Table 2: the improvement of DeeprETANet's MAE over the routing engine ETA's MAE, on the offline test set. With a routing engine MAE of 100 seconds (invented), that is 92.2 seconds (source-confirmed).

- Equation 15 defines it as the routing engine's mean absolute error minus the model's, divided by the routing engine's.

Q What does 2.9% measure?

A Table 7: improvement over the production XGBoost model in an online test, 2.91% MAE for delivery and 2.66% for rides, aggregated from multiple launches.

- Different baseline, different data and different setting. The two cannot be compared or added.

Q What do the ablations in Table 2 show?

A Without calibration, MAE improvement 7.73%. Without feature hashing, 7.41% with p50 5.10%. The p95 with and without hashing is the same, 7.99%.

- Both components contribute to the typical error. Hashing does not move the tail.

Q What does the split by type show?

A Table 3: delivery p50 2.26% and p95 11.05%; rides p50 6.25% and p95 7.62%. One average hides a median gain that is almost three times larger for rides, and a tail gain that is larger for delivery.

- Report the metric each team uses.

Q Reframe it.

A A percentage is meaningless without its baseline. Put the baseline and the data next to every number.

Basis: All table values and Equation 15 are from the paper (source-confirmed); the 100 second error is invented for illustration.

13 Review the loss function

CODE REVIEW

SCENARIO

A colleague implements the asymmetric Huber loss for training. After training, the model's ETAs run too early and delivery customers complain that couriers are often late. The configured omega is 0.3. Find the fault.

```
def asym_huber(y, y_hat, delta=60.0, omega=0.3):
    e = abs(y - y_hat)
    base = 0.5 * e * e if e < delta else delta * e - 0.5 * delta * delta
    w = omega if y > y_hat else 1.0 - omega
    return w * base
```

HOW TO THINK ABOUT IT

Work out which weight applies when the actual time is greater than the prediction, then price a late arrival.

Q What should the weights be?

A Omega applies when the actual is less than the prediction (overprediction), and one minus omega when the actual is greater (underprediction), per the paper's equation 14 (source-confirmed).

- With omega 0.3, underprediction, the late case, gets 0.7.

Q What does the code do?

A It gives omega to the case where the actual is greater than the prediction, so the late case gets 0.3 and the early case 0.7. The two weights are swapped.

- The model is then taught that early arrivals are the costly mistake.

Q Price an error of 120 seconds.

A The base loss is 5,400. Correct: late 3,780, early 1,620. Swapped: late 1,620, early 3,780.

- The swapped model prefers to predict early, which matches the complaints.

Q State the fix, and tests.

A Use omega when actual is less than predicted, and one minus omega otherwise.

- Tests: with omega 0.3 and delta 60, an error of 120 s costs 3,780 when the actual is later and 1,620 when it is earlier. The loss is continuous at an error of delta.

Q Reframe it.

A A sign or direction error in a loss is silent. Test each branch of a custom loss with hand-computed values before training on it.

REFERENCE CODE

```
def asym_huber(y, y_hat, delta=60.0, omega=0.3):
    e = abs(y - y_hat)
    base = 0.5 * e * e if e < delta else delta * e - 0.5 * delta * delta
    w = omega if y < y_hat else 1.0 - omega     # omega on overprediction (arrived early)
    return w * base
```

Basis: The loss and the direction of the weights are from the paper (source-confirmed); the numbers are illustrative arithmetic (recomputed); the fix was checked by hand and not run.

PART 4

Serving, evaluation and operations

The request path, two figures that do not agree, comparing a model with an old one and with a base system, and watching for drift.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Prediction service	A service that hosts a trained model and answers requests with its output. In the blog it is the Michelangelo online prediction service.
uRoute	The blog's name for the service that is the front end for all routing lookups and calls the routing engine and the model.
Clamp and ReLU on the output	A ReLU keeps the ETA from going negative. A clamp limits extreme values so one bad prediction cannot be absurd.
Fallback	A simpler answer used when the main path fails. Here the routing engine ETA could serve as one.
Mean residual	The average of the actual minus the predicted time. Near zero means the predictions have no systematic bias.
Drift	A gradual change in the data or in an input system that makes a model trained earlier less accurate.
Retraining cadence	How often a model is retrained. The paper's is weekly, and continuous training is listed as future work.
Metric noise	Random variation in a measured number. It falls with the square root of the sample size.
Where to read	Hu, Binaykiya, Frank, Cirit: DeeprETA: An ETA Post-processing System at Scale (arXiv:2206.02127) | Uber Engineering: DeepETA: How Uber Predicts Arrival Times Using Deep Learning (blog, February 2022)

14 Trace one ETA request

SERVING

SCENARIO

A product manager asks what happens, and in what order, when an app shows an arrival time, and what the team needs to keep running. Trace it using the blog.

HOW TO THINK ABOUT IT

Follow the blog's path, then ask what depends on what.

Q What is the path?

A Requests are routed through services to uRoute, the front end for all routing lookups. It asks the routing engine for route lines and ETAs, then calls the online prediction service with that ETA and other model features to get DeepETA's prediction (source-confirmed).

- Retraining and validation run as periodic workflows.

Q What happens to the model's output?

A It is a residual. It is added to the routing engine ETA, passed through a ReLU so the ETA is positive, and clamped to reduce the effect of extreme values (source-confirmed).

- The cap of three hours in a sketch is an illustrative choice.

Q What hardware?

A Production requests are served by a fleet of servers with 4 cores per host. Training used Nvidia Quadro RTX 5000 GPUs. A model that is 99% embedding lookups and about 0.5 million dense weights runs well on CPU.

- At the illustrative peak of 13,200 a second and 3.25 ms, about 43 requests are in flight and about 11 hosts at full use.

Q What does it depend on?

A The routing engine, since the residual needs its ETA. That dependency exists anyway because the route is shown to the user.

- The sources do not describe what happens when the prediction service is down. A fallback to the routing engine ETA is possible, and is this document's inference.

Q Reframe it.

A Draw the request path with its dependencies, and for each ask what is shown when it fails.

Basis: The path, the output processing and the hardware are from the sources (source-confirmed); the cap, the peak rate and the host count are illustrative; the fallback is reasoned.

15 Two sources, two figures

SOURCES

SCENARIO

The blog says any one prediction touches roughly 0.25% of the model's parameters. The paper's Table 4 says 0.5 million of 141.2 million parameters are outside the embeddings. A reviewer asks which is right. Work it out, and say what you would write.

HOW TO THINK ABOUT IT

Convert both to parameter counts and compare.

Q What does each say, in counts?

A 0.25% of 141.2 million is about 353,000 parameters touched. The dense part is 0.5 million, which is 0.35% of 141.2 million.

- A prediction has to use all of the dense weights, so it touches at least 0.5 million.

Q Do they conflict?

A As stated, yes: 353,000 is less than 500,000. The dense layers alone exceed the blog's figure.

- Both could be true of different configurations, or one is rounded loosely. Neither source reconciles them.

Q What do the sources agree on?

A The embeddings are the large majority of the parameters: the paper says more than 99%, and about 140 million parameters for the three networks it lists (source-confirmed).

- A request reads a few rows of each table and the dense layers.

Q What would you write?

A That the model has 141.2 million parameters of which 0.5 million are dense (the paper's table), that a request touches a small fraction of the embeddings, and that the blog's 0.25% cannot be reconciled with the table.

- Report both, name each source and do not merge them.

Q What does the size mean for operations?

A In float32 (assumed) the model is 565 MB. A weekly push to 36 hosts is about 20 GB of transfer, which is small.

- Memory per host is what matters, and 565 MB fits easily.

Q Reframe it.

A When two sources disagree, say so, give both numbers and use the one that is closer to the definition you need.

Basis: The 0.25%, the table values and the 99% statement are from the sources (source-confirmed); the 565 MB and 36 hosts assume float32 and 30% use; arithmetic recomputed.

16 Compare with the old model, and region by region

EVALUATION

SCENARIO

A VP asks whether the new model is better than the old one, and whether it is better everywhere. Answer with the shadow test and the city table, and say what the cities show.

HOW TO THINK ABOUT IT

Use Table 7 for the old model, and Table 6 for the regions, with the gaps between models.

Q How was the new model compared with the old one?

A In an online shadow test over 2 weeks, run globally. The launch criterion was to improve ETA accuracy over the online XGBoost model, measured by MAE, p50 and p95 across mega-regions (source-confirmed).

- A shadow test logs the new model's predictions without showing them to riders.

Q What were the results?

A Delivery: MAE +2.91%, p50 +4.74%, p95 +3.92%. Rides: +2.66%, +4.15%, +3.23% (Table 7).

- They are aggregated from multiple launches, and the baseline is the production XGBoost model.

Q Is it better everywhere?

A Table 6, MAE improvement over the routing engine: New York City 7.20%, Los Angeles 6.80%, Mexico City 6.60%, London 4.80%.

- It is better in all four, and London gains the least. The paper gives no cause.

Q What do the gaps show?

A London is lowest for every model: XGBoost 1.39%, ResNet 4.09%, HammockNet 4.28%. DeeprETANet's lead over XGBoost is 3.4 points in London, 3.1 in New York City, 2.0 in Los Angeles and 2.7 in Mexico City.

- So the low London number is not specific to the new model. Its relative gain over the old one is the largest there.

Q Reframe it.

A Ask which baseline and which region before you say better. A smaller gain over a base system can sit beside a larger gain over the old model.

Basis: Tables 6 and 7 and the shadow test are from the paper (source-confirmed); the gaps are arithmetic (recomputed).

17 Watch for drift

MONITORING

SCENARIO

The model is retrained weekly. The routing engine team plans to change how segment times are estimated. Describe what to monitor and how the two changes interact.

HOW TO THINK ABOUT IT

Think about what the residual depends on, and what the paper offers as a signal.

Q Why does a routing engine change matter?

A The model predicts the gap between the routing engine ETA and reality. If the routing engine changes, the gap changes, and a model trained on the old gap is wrong in a new way until it is retrained.

- This follows from the design. The paper says the approach is decoupled from the details of a routing engine, and tested only one.

Q What signal does the paper give?

A Figure 7 shows that after post-processing the mean residual is closer to 0. Tracking the mean residual over time, by city and request type, is a monitor of this kind (the monitoring use is this document's suggestion).

- A drift away from zero in one place is a flag.

Q What else would you watch?

A MAE, p50 and p95 by city and request type against the previous model, the share of clamped predictions, and the difference between the routing engine ETA and the corrected ETA.

- The clamp makes extreme predictions bounded and rare. A rise in clamping is a sign of trouble.

Q How does weekly retraining help?

A It bounds how long a shift goes uncorrected: up to about a week. The blog lists continuous, incremental training as future work.

- For a planned routing change, retrain and validate before and after, and consider a shadow period.

Q Reframe it.

A A model that corrects another system is coupled to it. Watch the correction itself, and coordinate releases.

Basis: The mean-residual observation, the weekly retraining and the future work are from the sources (source-confirmed); the monitoring plan and the coupling argument are reasoned.

18 Risks, and ten times the traffic

RISK

SCENARIO

A launch review asks what can go wrong with a post-processing ETA model and what grows if requests grow ten times. Use the sources and the earlier sizing.

HOW TO THINK ABOUT IT

List the failure modes, then scale the quantities and sort them by what they follow.

Q What can go wrong?

A Re-routing and noisy data, which the paper gives as the reason the routing engine ETA is off. Weak regions, as in London. Drift between weekly retrains. A single loss asymmetry that is wrong for some riders (reasoned).

- The paper's limitation: it was tested only with Uber's own routing engine.

Q What does ten times the requests do?

A Hosts scale with the request rate: about 11 become about 110 at full use, since each request is a short CPU call. The model size does not change.

- Unless one user request triggers more model calls, which would multiply it again.

Q What does it do to training?

A Weekly retraining reads ten times the requests, so training data and time grow, while the embedding tables stay the same size.

- The tables depend on vocabularies and hash bins, not traffic.

Q What gets easier?

A Measurement. Metric noise falls with the square root of the sample, about 3.2 times, so smaller gains and smaller regions become readable.

- Region-level and type-level numbers can then be tracked on shorter windows.

Q Reframe it.

A Sort the quantities by what they follow: traffic, vocabulary or number of regions. Each needs a different response.

Basis: The failure causes and the limitation are from the paper (source-confirmed); the 10 times figures are illustrative arithmetic; the loss-asymmetry risk is reasoned.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #7. The sources are the DeeprETA paper (arXiv:2206.02127) and the Uber Engineering blog post, both read in full on 2026-10-17, the post through a Wayback Machine snapshot because the original address no longer resolves. They give request volume only in words and do not publish the loss parameters, the number of features or a total latency budget, so answers about those are design arguments made for this book and were not run on real systems.

- Hu, Binaykiya, Frank, Cirit: DeeprETA: An ETA Post-processing System at Scale, arXiv:2206.02127 (June 2022), full text read 2026-10-17
https://arxiv.org/abs/2206.02127

- Uber Engineering: DeepETA: How Uber Predicts Arrival Times Using Deep Learning, February 10, 2022 (Wayback Machine snapshot, full text read 2026-10-17)
https://web.archive.org/web/20220701063549/https://eng.uber.com/deepeta-how-uber-predicts-arrival-times/